Expt No: 6

Date: 6/8/24 ERROR DETECTION & CORRECTION - Hamming code.

Aim: To write a program to implement error detection & Correction using Hamming Code.

Algorithm:

1) Start the Program.
2) Read the number of 4 bits data blocks (n) forming the input data Stream.
3) For each 4 bit data block D = d1 & d2, d3, d4, perform Hamming encoding.
   a) Arrange 7 bits with parity bits at position 1, 2, and 4
   b) Compute the parity bits using even parity.
      P1 = d1 XOR d2 XOR d4 (Cover bit - 1, 3, 5, 7)
      P2 = d1 XOR d3 XOR d4 (Cover bit - 2, 3, 6, 7)
   c) Form the 7 bit hamming code word: P1 P2 d1 P4 d2 d3 d4.
4) Simulate transmission by introducing a single bit error at a chosen bit position in the Code word.
5) At receiver, recompute the Check bits (Syndrome) from received Code word:
      C1 = P1 XOR d1 XOR d2 XOR d4
      C2 = P2 XOR d1 XOR d3 XOR d4
      C4 = P4 XOR d2 XOR d3 XOR d4
6) Compute Error position = C4 C2 C1 (binary no formed)
7) If error position = 0, no error has occured.
8) Extract the original 4 data bits from Corrected 7 bit code
9) Repeat Step 3 to 8 for all n blocks in the data Storage.

Output:

```
=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===
Enter number of 4-bit data blocks in the stream : 2

--- Block 1 ---
Enter 4 data bits (space separated, eg. 1 0 1 1): 0 1 0 1
Original Data bits : 0101
Encoded Hamming code (7 bits) : 0100101

Enter bit position to introduce error (1-7, 0 = no error): 3
Transmitted Code (with error induced at pos 3): 0110101
Received Code : 0110101
Checking Code : 0100101
Recovered Data Bits : 0101

--- Block 2 ---
Enter 4 data bits (space separated, eg. 1011): 1 1 0 1
Original Data Bits : 1101
Encoded Hamming Code (7 bits) : 1010101
Enter bit position to introduce error (1-7, 0 = no error): 5
Transmitted Code (with error induced at pos 5): 1010001
Received Code : 1010001
Checking for error...
Error detected at bit position : 5 → Correcting...
Corrected Code : 1010101
```

10) Stop the program.

Program:

```
def encode(d):
    """
        d: list of 4 bits [d1, d2, d3, d4] → return 7 bit
        Hamming Code List position (1-indexed) 1=p1, 2=p2, 3=d1
        4=p4, 5=d2, 6=d3, 7=d4 """
    h = [0]*7
    h[2] = d[0]  # position 3
    h[4] = d[1]  # position 5
    h[5] = d[2]  # position 6
    h[6] = d[3]  # position 7

    h[0] = h[2] ^ h[4] ^ h[6]  # p1 Covers 1, 3, 5, 7
    h[1] = h[2] ^ h[5] ^ h[6]  # p2 Covers 2, 3, 6, 7
    h[3] = h[4] ^ h[5] ^ h[6]  # p4 Covers 4, 5, 6, 7
    return h

def detect and Correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]
error_pos = c1*1 + c2*2 + c4*4  # Syndrome → faulty bit position

if error_pos != 0
        print ("Error detected at bit position : " error_pos)
        h[error_pos-1] ^= 1

else : print ("No error detected")
```

```
    return error_pos

def to_str(bits):
    return "".join(str(b) for b in bits)

def main():
    n = int(input())
    for b in range(1, n+1):
        print(f"Block {b}:")
        d = list(map(int, input().split()))
        h = encode(d)
        print(f"original data bits : {to_str(d)}")
        print(f"Encoded Hamming Code (7 bits) : {to_str(h)}")
        pos = int(input("Enter bit to introduce error (1-7), 0 =
                        "No error"))
        if 1 <= pos <= 7:
            h[pos-1] ^= 1
            print(f"Transmitted code : {to_str(h)}")
        else:
            print(f"Transmitted code : {to_str(h)}")
        print("Received Code", f"{to_str(h)}")
        print("Checking error")
        detect error and correct(h)
        print(f"Corrected Code : {to_str(h)}")
    rd = [h[2], h[4], h[5], h[6]]
    print(f"recovered data bits : {to_str(rd)}")
if __name__ == "__main__":
    main()
```

Result: The program to implement error detection & correction using hamming code concept was Successfully written in python, executed and verified with a Sample input data Source. Stream.